

Ревью кода на C++

Правила для успешного прохождения code review

Соблюдайте [стандарт](#).

Пишите [документацию](#).

Критерии review

Ревью проводится по списку типичных антипаттернов, часто встречающихся в коде. Антипаттерны сгруппированы по четырем смысловым блокам, границы между блоками достаточно условны. Большинство перечисленных пунктов так или иначе присутствует в корпоративном стандарте разработки на C++. По мере накопления опыта проведения ревью список может быть расширен.

Блок 1: Структура кода

Не минимизированы экспортируемые заголовочные файлы

Согласно стандарту разработки на C++ заголовочные файлы должны быть минимальными и самодостаточными. Под минимальностью понимается следующее: данный заголовочный файл должен подключать в себя лишь те файлы, без которых он не может быть использован. Т.е. не должно быть лишних `#include`-ов. Раздувание заголовочных файлов ведет к увеличению времени (пере)сборки проектов и увеличению объема отладочных символов для результирующего бинарника.

Экспортируемые заголовочные файлы не являются самодостаточными

Согласно стандарту разработки на C++ заголовочные файлы должны быть минимальными и самодостаточными. Под самодостаточностью понимается следующее: все символы, используемые в данном заголовочном файле, должны быть в каком-то виде в нем описаны. Критерий проверки простой: если данный заголовочный файл подключить в пустой `cpp`-файл и попробовать его скомпилировать, то сборка должна завершиться успешно. Чтобы данный инвариант поддерживался в коде, очень полезно бывает подключать `foo.hpp` первым `#include`-ом в `foo.cpp`. Несамодостаточность заголовочных файлов ведет к возникновению неожиданных ошибок компиляции клиентского кода.

Присутствуют лишние `#include`-ы в файлах с исходным кодом

В файлы с исходным кодом не должны подключаться лишние заголовочные файлы, т. е. файлы, сущности из которых не применяются в данной единице трансляции. Раздувание списка подключенных заголовочных файлов ведет к увеличению времени (пере)сборки проектов и увеличению объема отладочных символов для результирующего бинарника.

Огромные файлы с кодом

Код проекта должен быть организован таким образом, чтобы в нем было легко ориентироваться. Этому не способствуют файлы, содержащие тысячи строк исходного кода.

Блок 2: Оформление кода

Код оформлен не по стандарту разработки

Любое несоответствие стандарту разработки на C++. Пробелы, отступы, имена переменных и т.п.

Избыточная вложенность блоков кода

Избыточная вложенность блоков кода затрудняет его чтение и повышает вероятность совершения ошибки при его рефакторинге.

Предупреждения компилятора при сборке

Предупреждений при сборке быть не должно. Совсем и никаких. Используйте `Smake`-инструкцию `sbis_treat_warnings_as_errors()` для своих проектов.

Блок 3: Используемые языковые средства

Не используются «умные указатели» для управления динамической памятью

Использование операторов new/delete без умных указателей в современном C++ считается дурной практикой, ведущей к утечкам памяти и неустойчивости кода к возникновению исключений.

Не используется STL

Использование самопальных аналогов контейнеров и алгоритмов из состава STL ведет к появлению лишнего (как правило – багованного) кода и ухудшению читаемости.

Блоки вида catch(...)

Имеются в виду «подозрительные» случаи использования данного блока кода. Типа «посылаем запрос в БД и игнорируем все ошибки, не проверяя наличие ранее открытых на коннекте транзакций».

c-style cast

Согласно стандарту разработки должны использоваться операторы static_cast<>, reinterpret_cast<>, const_cast<> и dynamic_cast<>.

Константы в заголовочных файлах

Помещение констант, в особенности строковых, в заголовочных файлах ведет к раздуванию бинарного кода. Используйте ключевое слово extern при объявлении константы, ее определение располагайте в файле с реализацией.

Блок 4: Подозрительные конструкции в коде

Ручная сборка запросов

Тут обращается внимание на возможность осуществления SQL-инъекций при отсутствии экранирования строковых переменных перед вставкой в запрос.

Нет виртуального деструктора у абстрактного класса / класса с виртуальными функциями

Как правило, подобный код чреват утечками памяти при удалении объекта через указатель на базовый класс.

Copy-paste

Дублирование больших объемов исходного кода.

Огромные функции

Функции на N экранов, где N>5. Затрудняют чтение и отладку кода, повышают вероятность внести ошибку при рефакторинге.

Функции с кучей аргументов

Как правило, являются признаком плохой объектной декомпозиции. Подозрительными являются функции с более чем 10 аргументами.

Прочие странности

Прочие странные конструкции, не укладывающиеся в предыдущие группы. Всегда идет с пояснениями. Примеры из практики:

- Опечатки в именах переменных/классов/функций.
- Непотокобезопасная работа с глобальными переменными.
- Очевидно неэффективные алгоритмы (когда поддается локальному анализу).
- Странные макрос.
- Не к месту применяемые языковые средства (например, ключевое слово inline в cpp-файле).
- Мертвый код (большие блоки).
- Код, очевидно дублирующий классы из состава платформы.